

Tarea 2 Sistemas Distribuidos

Procesamiento, Resiliencia y Fallback con Apache Kafka

Dante Libiot, dante.libiot@mail.udp.cl
Benjamin Jimenez, benjamin.jimenez4@mail.udp.cl
Profesor: Nicolás Hidalgo

Junio, 2026

Índice

1. Introducción	3
2. Diseño de la Arquitectura	3
2.1. Descripción General	3
2.2. Componentes Principales	3
2.2.1. Cliente (Generador de Tráfico)	3
2.2.2. Broker de Mensajería (Apache Kafka)	4
2.2.3. Workers Consumidores (Generadores de Respuesta)	4
2.2.4. Almacenamiento de Estado y Métricas (Redis)	4
3. Implementación	4
3.1. Mecanismos de Fallback y Resiliencia	4
3.2. Escenarios de Fallo Controlado (Chaos Testing)	5
3.3. Estructura del Payload	5
4. Experimentos y Resultados	5
4.1. Decisiones de Diseño y Optimización	5
4.1.1. Cálculo Dinámico de Métricas Asíncronas	5
4.1.2. Dimensión de Memoria y Cuellos de Botella	5
4.2. Métricas de Evaluación	6
5. Análisis y Discusión	7
5.1. Desacoplamiento y Reinterpretación de la Latencia	7
5.2. Resiliencia y Semántica de Entrega (At-Least-Once)	7
5.3. Ingeniería de Caos y Aislamiento de Fallos (DLQ)	7
5.4. Escalabilidad Horizontal	7
6. Resultados Visuales	8
6.1. Procesamiento de Cargas Asíncronas y Comportamiento de la Caché	8
6.2. Ingeniería de Caos y Aislamiento de Fallos	11
6.3. Persistencia y Consistencia en los Tópicos de Apache Kafka	12
6.4. Escalabilidad Dinámica de Consumidores	13
7. Conclusiones	13
8. Referencias	14

1. Introducción

En este proyecto se implementó una plataforma distribuida para el análisis de consultas geoespaciales utilizando el dataset Google Open Buildings, con una capa de caché (Redis) que demostró reducir la latencia bajo patrones de tráfico asimétricos (Zipf). Sin embargo, dicha arquitectura presentaba una comunicación entre módulos puramente síncrona. Poco óptimo bajo escenarios de alta concurrencia o fallas temporales del servidor, el sistema pudo experimentar encolamiento por cuellos de botella, pérdida de consultas y degradación del rendimiento.

Esta etapa se basa en diseñar e implementar una evolución arquitectónica orientada a los eventos mediante la incorporación de **Apache Kafka**. El problema a resolver es el acoplamiento: el cliente no debe esperar a que el servidor termine de procesar para continuar inyectando tráfico. Para ello, se implementó la solución de introducir un sistema de colas (message broker) que separe el Generador de Tráfico de los Generadores de Respuestas (Workers).

Mediante esta arquitectura asíncrona, el sistema es capaz de tolerar picos masivos de carga almacenándolos en un *Backlog*, implementar tolerancia a fallos mediante políticas de reintento, y aislar mensajes irrecuperables a través de una **Dead Letter Queue (DLQ)** después de una cierta cantidad de intentos, garantizando resiliencia en un entorno distribuido.

2. Diseño de la Arquitectura

2.1. Descripción General

El sistema migra hacia una **arquitectura basada en eventos y microservicios desacoplados**. El flujo síncrono HTTP es reemplazado por un Productor-Consumidor apoyado por *topics* de mensajería.

La nueva topología de diseño opera de la siguiente manera: el Generador de Tráfico (Cliente) inyecta asíncronamente eventos JSON al tópico principal de Kafka a muy alta velocidad. Los *Workers* (consumidores independientes) extraen estos mensajes a su propio ritmo. Si el procesamiento de una consulta es exitoso, la respuesta se almacena en Redis y el consumidor confirma el mensaje (*commit*). Si ocurre un fallo en el procesamiento, el Worker enruta el mensaje a recuperación o aislamiento, manteniendo la estabilidad del sistema.

2.2. Componentes Principales

2.2.1. Cliente (Generador de Tráfico)

Actúa como inyector de tráfico. En lugar de hacer solicitudes HTTP como en la tarea pasada, utiliza `confluent_kafka.Producer` de manera asíncrona (*asyncio*). Permitiendo simular inyecciones de hasta 10.000 consultas en segundos sin bloquear el Event Loop local. Adicionalmente, cuenta con una interfaz web (Dashboard) que permite configurar patrones de distribución (Uniforme o Zipf) e inyectar fallos controlados (*Poison Pills*) para pruebas de ingeniería de caos (*Chaos Testing*).

2.2.2. Broker de Mensajería (Apache Kafka)

El centro de la arquitectura. Gestiona tres tópicos fundamentales:

- `queries-main`: Recibe el flujo principal de consultas desde el Productor.
- `queries-retry`: Sala de espera para mensajes que sufrieron fallos temporales de procesamiento y requieren un reintentó.
- `queries-dlq`: La *Dead Letter Queue*, destino final para mensajes irrecuperables.

2.2.3. Workers Consumidores (Generadores de Respuesta)

Microservicios distribuidos que usan de forma concurrente a los tópicos `queries-main` y `queries-retry` a través de *Consumer Group* (`gis-workers-group`). Estos consumidores antes de procesar la lógica, consultan a Redis. Si hay un *cache miss*, ejecutan el cálculo (simulando un retraso) e intentan guardar el resultado.

2.2.4. Almacenamiento de Estado y Métricas (Redis)

Este almacenamiento operar como caché LFU de 2MB limitados intencionalmente. Registra tasas de éxito, fallos, reintentos y evicciones en tiempo real, permitiendo que el Dashboard del Servidor calcule el *Throughput* y el *Backlog Size* sin consultar las APIs internas de Kafka, pues ambos reciben los mismos valores y el dashboard los visualiza.

3. Implementación

3.1. Mecanismos de Fallback y Resiliencia

Para garantizar una entrega *At-Least-Once* y evitar la pérdida silenciosa de mensajes frente a caídas imprevistas de un contenedor, se tomó la decisión de: **desactivar el auto-commit** en el consumidor de Kafka (`enable.auto.commit=False`).

El **offset** del mensaje solo avanza de forma explícita mediante un `consumer.commit(msg)` cuando ocurre una de las siguientes condiciones de cierre:

1. El mensaje fue procesado exitosamente y guardado en la caché.
2. El mensaje falló (teniendo un límite de 3 reintentos, para ofrecer un balance entre la tolerancia a fallos y la prevención de bloqueos prolongados en las particiones de kafka) y fue delegado asíncronamente al tópico de reintentos (`queries-retry`).
3. El mensaje superó el límite de reintentos y fue enrutado al tópico terminal (`queries-dlq`).

3.2. Escenarios de Fallo Controlado (Chaos Testing)

Para evaluar la tolerancia a fallos, se implementó una **Poison Pill determinística**. A través de la interfaz del Cliente, el evaluador puede inyectar tráfico malicioso. Cuando el Productor recibe esta instrucción, genera *payloads* con condiciones forzadas que el modelo de negocio rechaza (específicamente, *zona "Z5"* y *confianza mínima mayor a 0,90*).

El *Worker* intercepta esta anomalía lanzando una excepción (`ValueError`) de forma controlada. Al capturar el error, el sistema incrementa el contador de intentos del payload, envía la telemetría a Redis (`stats:retry_rate`), reencola el mensaje en Kafka, e intenta procesarlo nuevamente hasta enviarlo a la DLQ (`stats:dlq_rate`) cuando pase el límite de intentos.

3.3. Estructura del Payload

Para garantizar trazabilidad distribuida, cada consulta inyectada a Kafka respeta el siguiente JSON:

Campo	Tipo	Descripción
id	STRING	Identificador único UUIDv4
timestamp	FLOAT	Tiempo de origen en milisegundos
tipo	STRING	Tipo de consulta (Q1–Q5)
intentos	INT	Contador de fallos (inicializado en 0)
data	OBJECT	Parámetros específicos (<i>zona</i> , <i>confidence</i>)

Tabla 1: Estructura del mensaje JSON inyectado en Kafka.

4. Experimentos y Resultados

4.1. Decisiones de Diseño y Optimización

4.1.1. Cálculo Dinámico de Métricas Asíncronas

En una arquitectura síncrona, el cronómetro del experimento termina cuando el cliente recibe el último *HTTP 200*. En este esquema asíncrono, el cliente termina de inyectar 5.000 mensajes en apenas unos segundos. Si el Dashboard detuviese el *polling* en ese instante, ignoraría el trabajo en segundo plano de Kafka.

Por ello, se rediseñó el backend del servidor para que el experimento se considere “En Curso” mientras el **Backlog Size** sea mayor a cero. El *Backlog* se calcula algorítmicamente restando los mensajes procesados exitosamente y los mensajes en la DLQ al total proyectado por el inyector, es decir, el conjunto de mensajes procesados ya sea exitoso o fallido.

4.1.2. Dimensión de Memoria y Cuellos de Botella

Se restringió la memoria máxima de Redis intencionalmente a **2MB** con una política `allkeys-lfu`. Asimismo, dentro de la lógica del *Worker*, se introdujo un *sleep* aleatorio de

entre 0.2 y 0.5 segundos para simular una carga de base de datos (PostGIS). Esta configuración fuerza al clúster a experimentar *Lag* visible y presión de desalojos (*Evictions*), permitiendo evaluar el comportamiento bajo estrés real”.

4.2. Métricas de Evaluación

A las métricas de la etapa anterior (Hit rate, Throughput, Evictions) se incorporaron los siguientes indicadores:

Métrica	Definición
Backlog size	Cantidad de mensajes pendientes en Kafka (target - procesados).
Retry rate	Consultas reenviadas a tópicos de reintento.
Recovery rate	Consultas recuperadas exitosamente tras fallos temporales.
DLQ rate	Consultas enviadas a la Dead Letter Queue.
Recovery time	Tiempo transcurrido desde el primer fallo hasta que la cola de mensajes se vacía (Backlog = 0).

Tabla 2: Nuevas métricas de resiliencia.

5. Análisis y Discusión

El despliegue de la arquitectura basada en eventos con Apache Kafka expuso diferencias fundamentales frente a su predecesor síncrono, requiriendo un análisis profundo de las métricas obtenidas y de las decisiones de diseño adoptadas.

5.1. Desacoplamiento y Reinterpretación de la Latencia

Durante las pruebas de carga (inyección de 5.000 consultas en unos segundos), se observó un aumento extremo en la latencia, registrando valores de $p50 \approx 304$ ms y $p95 \approx 575$ ms [contrastados con las latencias de la tarea 1 hubo un aumento significativo por el efecto de cola]. En esta arquitectura distribuida la latencia total se define como la suma del tiempo en cola (*Backlog*) y el tiempo de procesamiento. El elevado $p95$ no refleja una degradación del *Throughput*, sino el tiempo máximo que el final de los mensajes debió esperar en el clúster. Kafka absorbió exitosamente el choque de tráfico masivo, priorizando la integridad de los datos sobre la inmediatez. Y por ello, la carga masiva de mensajes se encolan para su posterior procesado.

5.2. Resiliencia y Semántica de Entrega (At-Least-Once)

La decisión de configurar el parámetro `enable.auto.commit = False` en los consumidores probó ser un cambio crítico para garantizar la tolerancia a fallos. Frente a caídas abruptas de un contenedor *Worker*, la confirmación manual de *offsets*, la cual fue condicionada a la persistencia exitosa en Redis o al enrutamiento a colas secundaria, demostró mantener una *At-Least-Once*, garantizando que ningún evento generado por el cliente se pierda silenciosamente, evidenciando que ocurre con cada mensaje.

5.3. Ingeniería de Caos y Aislamiento de Fallos (DLQ)

La inyección de la *Poison Pill* (eventos preconfigurados para fallar algorítmicamente a propósito) demostró el comportamiento del sistema bajo condiciones adversas controladas. El mecanismo de recuperación operó de la siguiente manera: reencolando los mensajes (*queries-retry*) hasta el límite de `MAX_RETRIES = 3`. Posteriormente, se activó la *Dead Letter Queue* (DLQ) por superar los reintentos permitidos, encapsulando la "basura" informacional y permitiendo que el *Recovery Time* del clúster se detuviera exitosamente una vez vaciado el *Backlog* con los mensajes en cola.

5.4. Escalabilidad Horizontal

En el experimento se probó un escalamiento dinámico (`--scale worker=3`). Al compartir el mismo `group.id`, Kafka rebalancea automáticamente las particiones del tópico `queries-main`. El resultado fue una aceleración proporcional del *Throughput* y una caída

pronunciada del *Backlog Size*, confirmando que el cuello de botella impuesto por un único nodo se mitiga distribuyendo la carga sin necesidad de detener el clúster.

6. Resultados Visuales

La presente sección consolida la evidencia recolectada para validar el diseño, corroborando el cumplimiento de los requerimientos como resiliencia, tolerancia a fallos, manejo asíncrono de sobrecargas y efectividad de la memoria caché.

6.1. Procesamiento de Cargas Asíncronas y Comportamiento de la Caché

Para evaluar el sistema, se inyectaron cargas masivas bajo distribuciones Uniforme y Zipf.

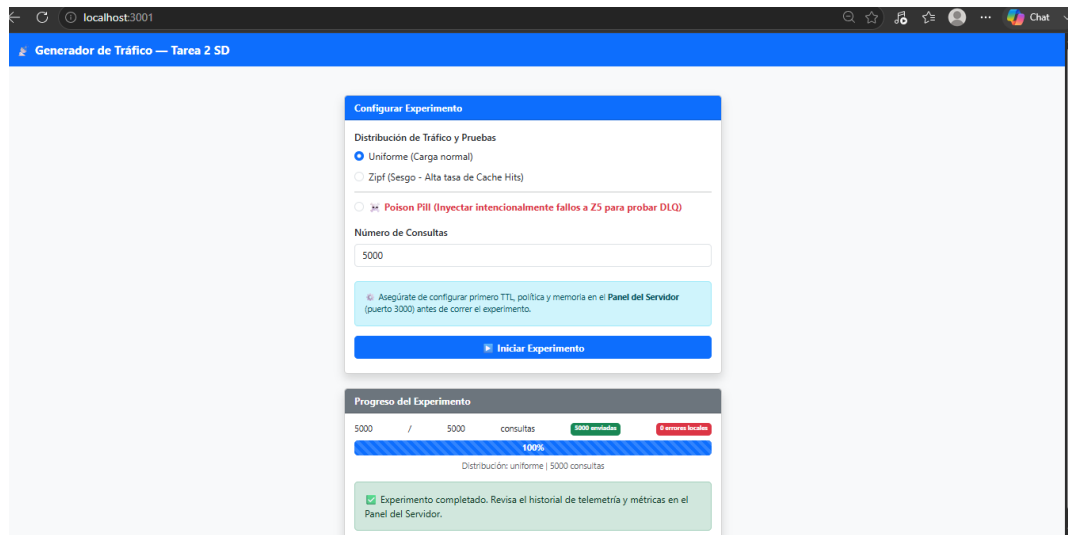


Figura 1: Telemetría tras 5.000 consultas (Uniforme)

6.1 Procesamiento de Cargas Asíncronas y Comportamiento de Resultados VISUALES



Figura 2: Evidencia el *Hit Rate* y la activación de evicciones en Redis.

La Figura 2 (Uniforme, 5.000 consultas) muestra un bajo *Hit Rate* que a pesar del tiempo se fue regulando a (9,94 %) y una alta latencia *p95* (49542 ms en aumento). Este valor refleja el tiempo de espera acumulado en el *Backlog* de Kafka, confirmando la absorción asíncrona de la carga.

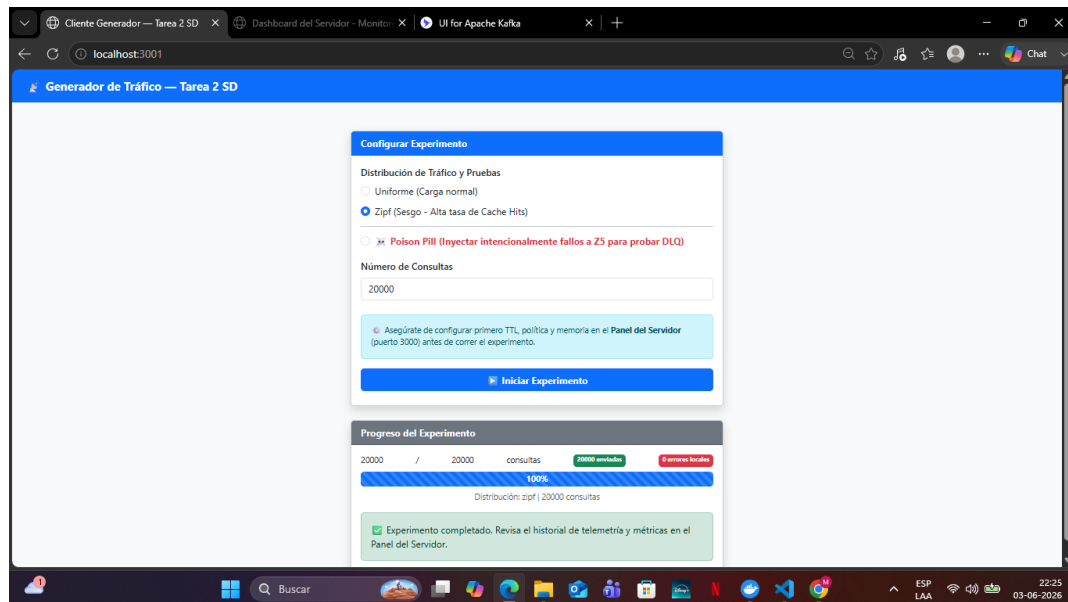


Figura 3: Telemetría tras 20.000 consultas (Zipf).

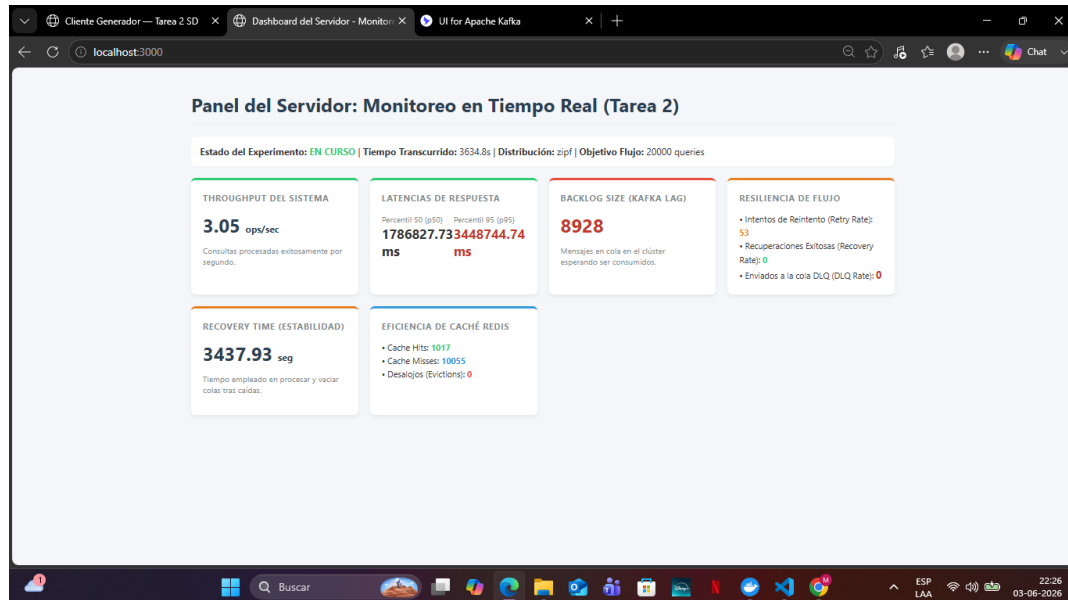


Figura 4: Evidencia el *Hit Rate* y la activación de evicciones en Redis.

En contraste, la Figura 3 (Zipf, 20.000 consultas) evidencia la eficacia de la caché: el *Hit Rate* asciende y se registran las evicciones, confirmando el correcto funcionamiento de la política `allkeys-lfu` bajo el estricto límite de 2MB. Cabe mencionar que al tomar las capturas la gran latencia que existe es provocado dado que habian más peticiones por detrás procesandoce, lo que aumento significativamente su valor.

6.2. Ingeniería de Caos y Aislamiento de Fallos

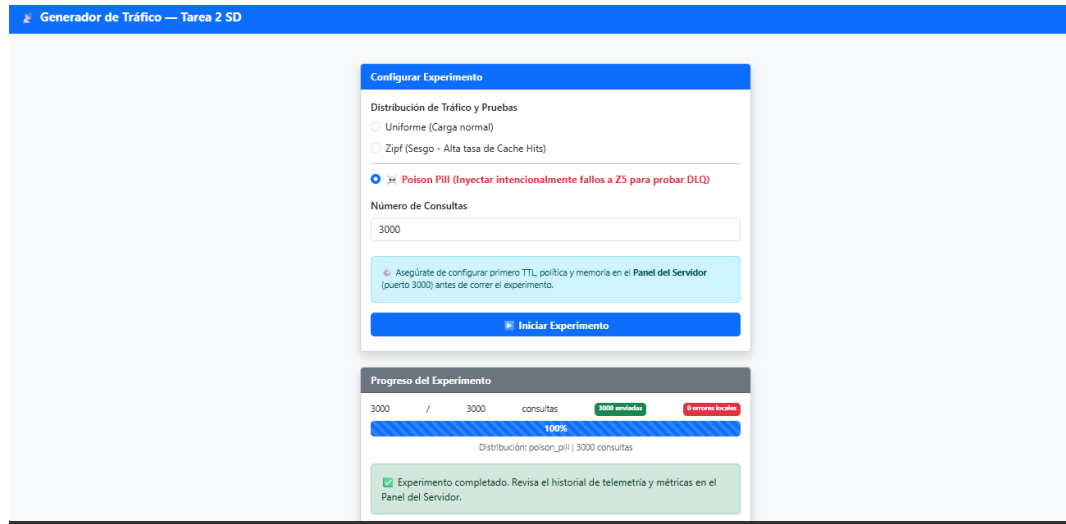


Figura 5: Métricas de resiliencia tras inyectar 3.000 *Poison Pills*. Se cuantifica la derivación a colas de reintento y su segregación en la DLQ.



Figura 6: Métricas con 3 workers y posion pill activada

La tolerancia a fallos se evaluó inyectando 3.000 cargas maliciosas (*Poison Pills*) (Figura 5). Los *Workers* interceptaron los errores, generando exactamente 9.000 reintentos (*Retry Rate*, equivalente al límite de 3 reintentos por consulta). Al agotar este límite, las anomalías fueron aisladas en la *Dead Letter Queue* (*DLQ Rate* = 3.000). El sistema restableció su *Backlog* a cero en 1,391,80 segundos (*Recovery Time*), demostrando resiliencia estructural.

6.3. Persistencia y Consistencia en los Tópicos de Apache Kafka

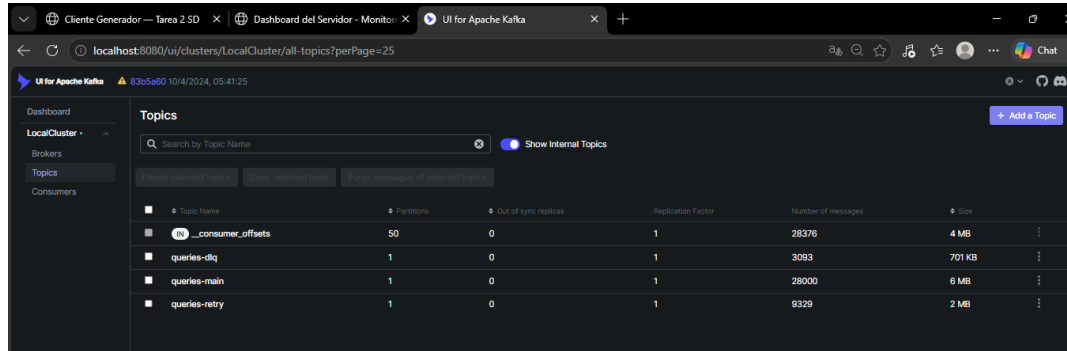


Figura 7: Interfaz Kafka UI exponiendo el acumulado histórico persistido en los tres tópicos.

La Figura 7 corrobora la consistencia del *broker*. Los volúmenes reflejan el acumulado de las tres pruebas: *queries-main* persistió el histórico total de 28.000 mensajes (5.000 Uniforme + 3.000 Poison + 20.000 Zipf); *queries-retry* procesó los 9.000 reintentos; y *queries-dlq* almacenó los 3.000 mensajes aislados.

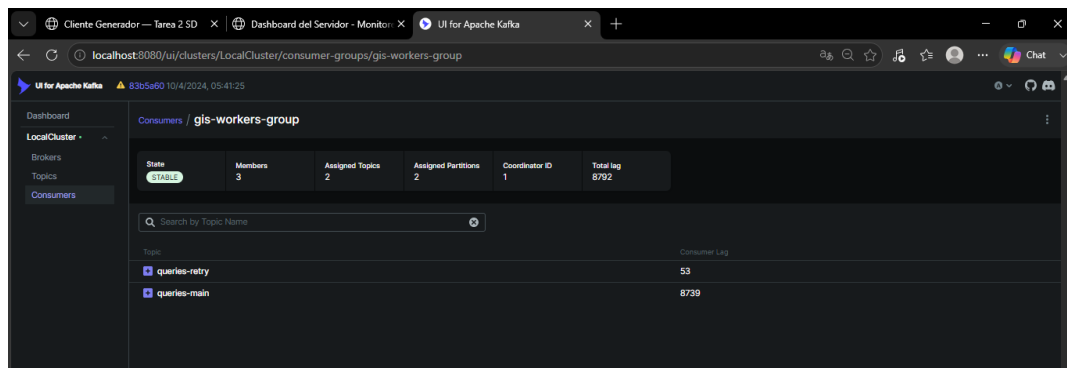


Figura 8: Cargas útiles en *queries-retry*, evidenciando el avance iterativo del contador de intentos.

Las Figuras 8 y 7 confirman que los *payloads* actualizaron sus contadores internamente antes del aislamiento, validando el correcto manejo manual de *offsets* (`enable.auto.commit = False`).

6.4. Escalabilidad Dinámica de Consumidores

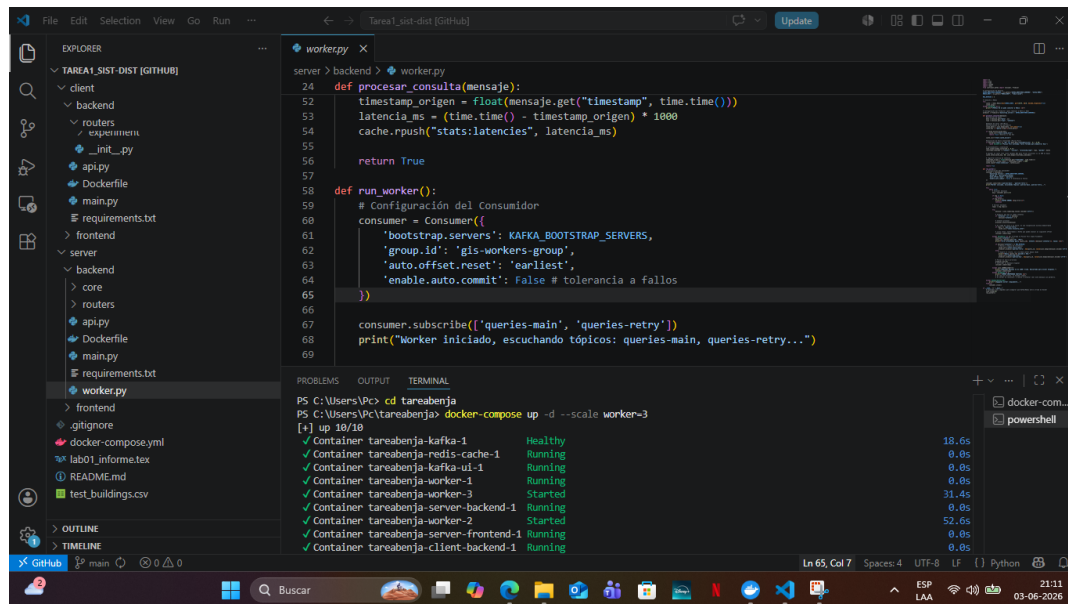


Figura 9: Comando Docker para la adición de réplicas de contenedores *Worker* en caliente.

Las Figuras 9 demuestran el escalamiento horizontal dinámico. Al solicitar tres réplicas de *Workers*, estas se acoplaron exitosamente al `gis-workers-group`. Kafka rebalanceó automáticamente las particiones, distribuyendo la carga concurrente y optimizando el *Throughput* global sin interrupciones del servicio.

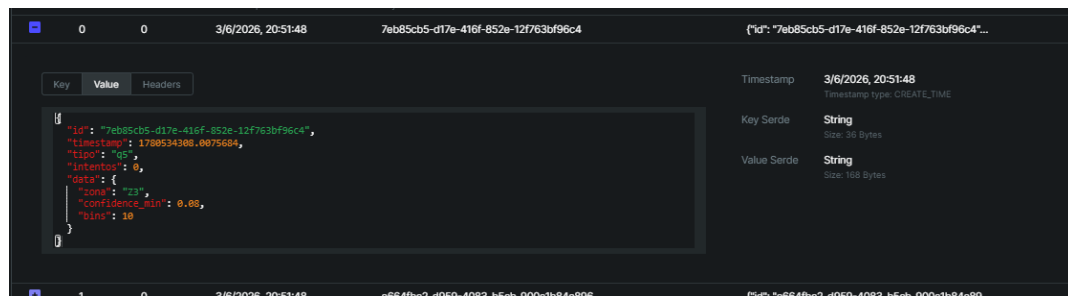


Figura 10: Indexación de las consultas que llegan a Kafka (Formato Json)

7. Conclusiones

La migración de la arquitectura síncrona a un modelo asíncrono basado en Apache Kafka cumple bien con el objetivo de desacoplar la plataforma. El sistema demostró ser elástico, escalable y resiliente.

A partir del análisis experimental, se establecen las siguientes conclusiones:

- **Manejo de Picos de Tráfico:** La arquitectura de colas es efectiva para proteger la capa de datos. Mientras el Productor inyecta cargas masivas de peticiones en menos de dos segundos, Kafka absorbe el choque (formando el *Backlog*), permitiendo a los *Workers* operar de manera estable según sus capacidades de I/O y a sus tiempos.
- **Eficacia del Aislamiento:** La implementación de una Dead Letter Queue, combinada con políticas de reintento, logró segregar el tráfico corrupto o fallido sin que ningún componente detuviera su ejecución, demostrando estabilidad arquitectónica a través del *Chaos Testing*.
- **Observabilidad:** La recolección de métricas desacopladas y su visualización en tiempo real es indispensable. Sin la capacidad de calcular dinámicamente el *Backlog Size* y el *Recovery Time*, sería imposible discernir si un sistema asíncrono falló silenciosamente o si simplemente se encuentra procesando la cola en segundo plano.

En síntesis, se logró la construcción de un sistema distribuido tolerante a fallos que gestiona recursos de forma eficiente bajo estrés, integrando con éxito la mensajería asíncrona, bases de datos en memoria y escalamiento basado en microservicios.

8. Referencias

- Google Open Buildings Dataset: <https://sites.research.google/gr/open-buildings/>
- Apache Kafka Documentation: <https://kafka.apache.org/documentation/>
- Redis Documentation: <https://redis.io/docs/>
- Docker y Docker Compose Documentation: <https://docs.docker.com/>

Links

1. Repositorio de Github: https://github.com/B3-njamin/Tarea1_sist-dist.git
2. Video de funcionamiento: https://drive.google.com/drive/folders/1Pkh_LShYw07PkBlw68yQusp=drive_link